Phishing Website Detection Using Machine Learning

Matthew Olivo, Yesid A. Cardenas and Naledge Killins

MS AAI, University of San Diego

AAI-500: Applied Probability and Statistics for AI

Professor Leonid Shpaner

June 22, 2026

Phishing Website Detection Using Machine Learning

Abstract

Phishing attacks remain one of the most common cybersecurity threats, targeting users through fraudulent websites designed to steal sensitive information. Detecting phishing websites automatically is an important application of machine learning because many phishing indicators can be extracted directly from website URLs and metadata. This project investigates the use of supervised machine learning techniques to classify websites as either legitimate or phishing. Using the UCI Phishing Websites Dataset, multiple machine learning models were trained and evaluated, including Logistic Regression, Decision Tree, and Random Forest classifiers. Exploratory data analysis was conducted to identify the most informative features associated with phishing behavior. Results indicate that the Random Forest classifier achieved the highest overall performance with an accuracy of approximately 94.4% and an F1 score of 0.942. Feature importance analysis revealed SSL Final State, URL of Anchor, Web Traffic, and Prefix/Suffix usage as the strongest predictors of phishing activity.

Introduction

Phishing websites are malicious webpages designed to impersonate legitimate organizations and trick users into revealing sensitive information such as passwords, banking credentials, and personal data. Cybercriminals often exploit user trust by creating convincing replicas of popular websites. As phishing attacks continue to increase in sophistication and scale, automated detection methods have become critical for cybersecurity defense.

Machine learning provides a promising solution because phishing websites exhibit measurable characteristics that distinguish them from legitimate websites. Features such as SSL certificate status, URL structure, domain registration information, and website traffic patterns can be used to train predictive models capable of identifying phishing attempts.

The purpose of this project is to evaluate the effectiveness of machine learning algorithms for phishing website detection and determine which website characteristics provide the greatest predictive value.

Dataset Description

The dataset used in this project was obtained from the UCI Machine Learning Repository Phishing Websites Dataset. The dataset contains 11,055 website observations and 30 predictive features describing URL characteristics, domain information, traffic indicators, and webpage attributes.

The target variable, Result, classifies websites as:

- 1 = Legitimate Website
- -1 = Phishing Website

Initial class distribution analysis showed:

- Legitimate Websites: 6,157 (55.7%)
- Phishing Websites: 4,898 (44.3%)

The relatively balanced distribution reduces concerns regarding severe class imbalance and allows model performance metrics such as accuracy and F1-score to be interpreted reliably.

Data Cleaning and Preparation

Prior to model development, the dataset was inspected for missing values and duplicate observations. No missing values were identified. However, duplicate detection revealed 5,206 duplicate rows.

After removing duplicates, the dataset size decreased from 11,055 observations to 5,849 unique observations.

The final cleaned dataset contained:

- 5,849 observations
- 30 predictor variables
- Binary classification target

An 80/20 train-test split was performed using stratified sampling to preserve class distributions between training and testing sets.

Training Set: 4,679 observations

Testing Set: 1,170 observations

Exploratory Data Analysis

Exploratory data analysis (EDA) was performed to better understand the structure of the dataset and identify website characteristics associated with phishing behavior. The analysis consisted of two parts: (1) dataset-level analysis examining class distributions and feature correlations, and (2) detailed feature-level analysis of key phishing indicators.

EDA Part 1: Dataset Structure and Correlation Analysis

The dataset contains 11,055 website observations, consisting of 6,157 legitimate websites (55.7%) and 4,898 phishing websites (44.3%). The relatively balanced class distribution reduces concerns regarding class imbalance and allows standard evaluation metrics such as accuracy, precision, recall, and F1-score to be interpreted reliably.

A correlation heatmap was generated to examine relationships among all features. Most variables exhibited relatively low correlation with one another, suggesting limited redundancy and minimal risk of multicollinearity.

The strongest correlations with the target variable (Result) were:

Feature	Correlation
SSLfinal_State	0.715
URL_of_Anchor	0.693
Prefix_Suffix	0.349
web_traffic	0.346
having_Sub_Domain	0.298

These results indicate that SSL security characteristics, URL structure, and website popularity metrics are among the most informative indicators of phishing activity.

EDA Part 2: Feature Analysis

To further investigate phishing behavior, a detailed feature analysis was conducted on five attributes commonly cited in cybersecurity research as strong predictors of malicious websites: URL Length, SSL Final State, Prefix/Suffix Usage, IP Address Usage, and Web Traffic. These features capture both structural URL characteristics and broader network-level indicators that can distinguish phishing websites from legitimate domains.

URL Length

Long URLs were associated with substantially higher phishing rates. Attackers frequently create lengthy URLs to conceal the true destination, insert recognizable brand names within the address, or overwhelm users' ability to visually inspect the link. The analysis demonstrated a clear increase in phishing prevalence as URL length increased.

SSL Final State

SSL/TLS security status emerged as one of the strongest indicators of website legitimacy. Websites using trusted SSL certificates were overwhelmingly legitimate, while sites lacking valid certificates or relying on self-signed certificates were strongly associated with phishing activity. This finding aligns closely with the correlation analysis, where SSLfinal_State was the single most strongly correlated feature with the target variable.

Prefix/Suffix Usage

Domains containing hyphens were considerably more likely to be phishing websites. Attackers commonly insert words such as “secure” or “login” around brand names to create deceptive domain names that appear trustworthy while remaining distinct from legitimate domains. The data showed that domains containing hyphens were disproportionately associated with phishing behavior.

IP Address Usage

Websites using raw IP addresses instead of registered domain names were significantly more likely to be phishing sites. This reflects a common phishing tactic that avoids domain registration and allows attackers to rapidly rotate infrastructure when domains are blocked or detected. Although highly informative, correlation analysis suggests that SSL status and URL anchor characteristics provide even stronger predictive signals overall.

Web Traffic

Legitimate websites typically accumulate traffic over time and establish a measurable online presence. In contrast, phishing websites are often short-lived and receive little traffic before being removed. The analysis showed that low-traffic and unranked websites were much more likely to be phishing sites, while highly trafficked websites were predominantly legitimate.

Overall, both the correlation analysis and feature-level investigation identified SSL security status, URL anchor characteristics, web traffic, and domain structure as the most influential indicators of phishing behavior. These findings provide strong support for their importance in the machine learning models evaluated later in this study.

Model Development

Three supervised machine learning models were evaluated:

Logistic Regression

Logistic Regression was selected as a baseline linear classifier. Feature standardization was applied prior to training.

Decision Tree

A Decision Tree classifier with a maximum depth of six was implemented to capture nonlinear relationships while reducing overfitting.

Random Forest

A Random Forest ensemble consisting of 100 decision trees was used to improve predictive accuracy and model robustness through aggregation.

Model Evaluation

Model performance was evaluated using Accuracy, Precision, Recall, and F1 Score.

Model	Accuracy	Precision	Recall	F1 Score
Random Forest	0.944	0.939	0.945	0.942
Logistic Regression	0.913	0.900	0.922	0.911

Model	Accuracy	Precision	Recall	F1 Score
Decision Tree	0.909	0.889	0.929	0.908

The Random Forest classifier achieved the strongest overall performance across all evaluation metrics.

Feature Importance Analysis

Random Forest feature importance scores identified the most influential variables in website classification:

1. SSLfinal_State (0.310)
2. URL_of_Anchor (0.231)
3. having_Sub_Domain (0.064)
4. web_traffic (0.057)
5. Prefix_Suffix (0.053)

These findings align closely with the correlation analysis and feature-level EDA, reinforcing the importance of SSL configuration, URL structure, and website popularity indicators in phishing detection.

Conclusion

This study demonstrated the effectiveness of machine learning techniques for phishing website detection using structural website characteristics. After data cleaning and exploratory

analysis, three classification models were evaluated. Random Forest achieved the highest performance with approximately 94.4% accuracy and a 0.942 F1 score.

Feature analysis consistently identified SSL Final State, URL of Anchor, Web Traffic, and Prefix/Suffix usage as the strongest predictors of phishing activity. These findings suggest that machine learning systems can successfully leverage URL and website metadata to detect malicious websites before users interact with them.

Future work could incorporate additional real-time website attributes, modern URL embeddings, and deep learning techniques to further improve detection performance against increasingly sophisticated phishing campaigns.

References

Das Guptta, S., Shahriar, K. T., Alqahtani, H., Alsalman, D., & Sarker, I. H. (2022). Modeling hybrid feature-based phishing websites detection using machine learning techniques. *Annals of Data Science*, *11*(1), 217–242. https://doi.org/10.1007/s40745-022-00379-8

Mohammad, R. M., & McCluskey, L. (2012). Phishing Websites. *UCI Machine Learning Repository*. https://archive.ics.uci.edu/dataset/327/phishing+websites

Appendix

EDA Part 1: Dataset Overview

```
[1]: import pandas as pd
     df = pd.read_csv('dataset.csv')
     df['label'] = df['label'].map({'Phishing Dataset.arff': 'phishing', 'legitimate': 'legitimate'})
```

df.head()

```
[2]: df.head()
```

	having_IP_Address	URL_Length	Shortining_Service	having_At_Symbol	double_slash_redirecting	Prefix_Suffix	having_Sub_Domain	SSLfinal_State	Domain_registeration_length	Favicon	...	popupWindow	Iframe	age_of_domain	DNSRecord	web_traffic
0	-1	1	1	1	1	-1	-1	-1	-1	1	...	1	1	-1	-1	0
1	1	1	1	1	1	-1	0	0	-1	1	...	-1	1	-1	-1	0
2	1	0	1	1	1	-1	-1	-1	-1	1	...	1	1	1	-1	1
3	1	0	1	1	1	-1	-1	-1	-1	1	...	1	1	1	-1	1
4	1	0	-1	1	1	-1	1	1	-1	1	...	-1	1	-1	-1	0

5 rows × 31 columns

df.shape

```
[3]: df.shape
```

```
(11055, 32)
```

df['Result'].value_counts()

```
[4]: df['Result'].value_counts()
```

```
Result
 1    6157
-1    4898
Name: count, dtype: int64
```

df.isnull().sum()

```
[5]: df.isnull().sum()
```

```
having_IP_Address              0
URL_Length                     0
Shortining_Service             0
having_At_Symbol               0
double_slash_redirecting       0
Prefix_Suffix                  0
having_Sub_Domain              0
SSLfinal_State                 0
Domain_registeration_length    0
Favicon                        0
port                           0
HTTPS_token                    0
Request_URL                    0
URL_of_Anchor                  0
Links_in_tags                  0
SFH                            0
Submitting_to_email            0
Abnormal_URL                   0
Redirect                       0
on_mouseover                   0
RightClick                     0
popupWindow                    0
Iframe                         0
age_of_domain                  0
DNSRecord                      0
web_traffic                    0
Page_Rank                      0
Google_Index                   0
Links_pointing_to_page         0
Statistical_report             0
Result                         0
dtype: int64
```

Class Distribution: Phishing vs Legitimate

Within the set of observations, we can see that the number of legitimate websites minus the websites that are phishing. We get there by evaluating the dataset which is 11,055 websites large and parsing it for legitimate and phishing sites. To achieve this, we need to clear the data to the code can discern what values are legitimate websites, and which are phishing. From there, we understand that 6157 are legitimate sites, and 4898 are phishing. To display this I created a dataset bar chart that shows the classes are roughly balanced with legitimate websites slightly outweighing phishing websites. This feature can be analyzed as we do not need to remove any features.

Import statistics as sns

```
[6]: import statistics as sns
```

plt.figure(figsize=(20,15))

```
plt.figure(figsize=(20,15))
sns.heatmap(df.corr(), cmap='coolwarm', annot=False)
plt.title('Correlation Heatmap of All Features')
plt.show()
```

Correlation Heatmap of All Features

target_corr = corr['Result'].drop('Result') corr_values=pd.DataFrame(target_corr).sort_values(by='Result')

```
[7]: target_corr = corr['Result'].drop('Result')
     corr_values=pd.DataFrame(target_corr).sort_values(by='Result')
```

	Result
SSLfinal_State	0.7104
URL_of_Anchor	0.6862
Prefix_Suffix	0.5850
web_traffic	0.5389
having_Sub_Domain	0.5363
Request_URL	0.4089
Links_in_tags	0.3819
Domain_registeration_length	0.2239
SFH	0.2149
Google_Index	0.1930

Plot

```
[8]: plt.figure(figsize=(8,5))
     corr_values['Result'].sort_values(ascending=True).plot(kind='barh', color='teal')
     plt.title('Top 10 Features Correlated with Result')
     plt.xlabel('Correlation with Result')
     plt.tight_layout()
     plt.show()
```

Top 10 Features Correlated with Result

Considering the features are a mix of binary, three-state, and continuous values, I chose to use a correlation heatmap to quickly scan for and visualize strong positive correlations. The heatmap shows most features don't strongly relate to each other. The darkest red squares are the strongest correlations. We still need to look closer to see how the features relate to the target variable, so I also created a bar chart. This makes it easier to rank features by their correlation with Result. The strongest positive correlations are SSLfinal_State (0.71), URL_of_Anchor (0.69), Prefix_Suffix (0.59), and web_traffic (0.54). The weaker ones are near-neutral, like SFH (0.21) and Google_Index (0.19). Domain_registeration_length is the only feature with a negative correlation (−0.22), meaning longer registration periods are slightly more common in legitimate websites. I kept all features, including the weak ones, because even moderate signals can help the model. Domain_registeration_length, the shortest bar, is the only feature I'd consider dropping if I needed to reduce complexity.

EDA Part 2: Feature Analysis

The second observation I created is a grid of statistical analysis of the key features extracted from the Phishing Websites Dataset. Each feature was selected based on its theoretical relevance to phishing detection and its practical use in the cybersecurity literature. The goal is to examine how the distributions of each feature differ between **phishing** and **legitimate** websites, providing interpretable evidence for model selection in subsequent sections.

The grid includes seven features, each displayed as a pair of plots: a proportional distribution (left) and absolute counts (right). The features examined are **URL Length**, **SSL Final State**, **Prefix/Suffix Usage**, **IP Address Usage**, and **Web Traffic** (this applies to all 4, 6DS).

Encoding note: In this dataset, feature values follow the convention: `1` = legitimate indicator, `-1` = phishing indicator. The integer variable `Result` uses `1` for legitimate and `-1` for phishing.

The goal of this analysis is to evaluate how the distributions and statistical tests of these distinct traits and selected attributes differ between legitimate websites and phishing websites, to ensure informed decisions for the models. 6DS.

Code Cell: Feature Analysis and Visualization

```
[9]: import matplotlib.pyplot as plt
     import seaborn as sns
     import numpy as np
     from scipy.stats import chi2_contingency

     # 1. Feature Analysis: Proportional vs Absolute Counts
     # Create a figure with 7 rows and 2 columns
     fig, axes = plt.subplots(7, 2, figsize=(15, 35))

     # Define the features to analyze
     features = ['URL_Length', 'SSLfinal_State', 'Prefix_Suffix', 'having_IP_Address', 'web_traffic', 'having_Sub_Domain', 'Domain_registeration_length']

     for i, feature in enumerate(features):
         # Calculate proportions
         prop = df.groupby('Result')[feature].value_counts(normalize=True).unstack()
         prop.index = ['Phishing', 'Legitimate']

         # Plot Proportional Distribution
         prop.plot(kind='bar', ax=axes[i, 0], color=['#ff9999', '#66b3ff'])
         axes[i, 0].set_title(f'Proportional Distribution (%) Within Categories: {feature}')
         axes[i, 0].set_ylabel('Proportion')
         axes[i, 0].legend(['Phishing', 'Legitimate'])

         # Plot Absolute Counts
         counts = df.groupby('Result')[feature].value_counts().unstack()
         counts.index = ['Phishing', 'Legitimate']
         counts.plot(kind='bar', ax=axes[i, 1], color=['#ff9999', '#66b3ff'])
         axes[i, 1].set_title(f'Absolute Count: {feature}')
         axes[i, 1].set_ylabel('Count')
         axes[i, 1].legend(['Phishing', 'Legitimate'])

     plt.tight_layout()
     plt.show()

     # 2. Chi-Square Test for Statistical Significance
     print("Chi-Square Test Results:")
     for feature in features:
         contingency_table = pd.crosstab(df[feature], df['Result'])
         chi2, p, dof, expected = chi2_contingency(contingency_table)
         print(f"{feature}: p-value = {p:.4f}")
```

```
Chi-Square Test Results:
URL_Length: p-value = 0.0000
SSLfinal_State: p-value = 0.0000
Prefix_Suffix: p-value = 0.0000
having_IP_Address: p-value = 0.0000
web_traffic: p-value = 0.0000
having_Sub_Domain: p-value = 0.0000
Domain_registeration_length: p-value = 0.0000
```

Feature Analysis: URL Length Distribution

Feature Analysis: SSL/TLS Security States

Feature Analysis: Domain Hyphen (Prefix/Suffix) Usage

Feature Analysis: IP Address Usage in Domain Name

Feature Analysis: Global Web Traffic Rankings

How These Features Differ Between Phishing and Legitimate Websites

The feature analysis conducted in the section examined five URL and website-level characteristics known to be theoretically linked to phishing behavior. The empirical results from this dataset confirm and quantify those relationships.

URL Length

While short, clean URLs are overall predominantly legitimate, long web addresses are actually quite common for both classes. Because web platforms and search ads typically use long links, inserting a simple rule to block them would accidentally shut down valid business traffic. Instead, URL length is most powerful when paired with other signals, ultimately proving that machine learning is required to evaluate multiple risk factors at once to protect users accurately.

SSL Final State

The SSL Final State feature is the most powerful indicator we discovered. Legitimate websites are overwhelmingly trusted, while phishing sites rely heavily on untrusted or suspicious connections. This makes sense because attackers rarely go through the trouble of purchasing expensive, verified security certificates for their temporary, malicious domains. Ultimately, this feature is the strongest single predictor we have, giving us a clear, reliable way to separate safe traffic from dangerous threats.

Prefix/Suffix (Hyphen in Domain)

The presence of a hyphen in a domain name is a major phishing indicator. Attackers use this technique to mimic trusted domains while appending hyphens from a distinct addition — for example, inserting `paypal-secure-login` into a fake domain. This also shows that all legitimate domains do not use hyphens, making this one of the cleanest binary signals in the dataset.

IP Address Usage

Using a raw IP address instead of a domain name is a huge red flag. Legitimate companies invest in a domain, but attackers often use temporary, compromised servers to host their sites. Because most users rarely memorize numbers and instead rely on domain names to recognize trusted platforms, this feature is a highly effective way to flag suspicious behavior before anyone even clicks.

Web Traffic

Established, legitimate websites accumulate massive traffic over time. Phishing websites are typically launched — active for hours or days — and therefore have no meaningful traffic history. The data confirms that sites with low or no measurable traffic are predominantly phishing. This feature is particularly useful for identifying brand-new threats, because even a perfectly convincing fake website will struggle to gain any real visitors. Extreme values in this feature can be used to flag suspicious domains almost immediately.

Modeling

1. Import Libraries

```
[10]: import pandas as pd
      import numpy as np
      from sklearn.model_selection import train_test_split
      from sklearn.preprocessing import StandardScaler
      from sklearn.metrics import classification_report

      from sklearn.linear_model import LogisticRegression
      from sklearn.tree import DecisionTreeClassifier
      from sklearn.ensemble import RandomForestClassifier

      from sklearn.metrics import (
          accuracy_score,
          precision_score,
          recall_score,
          f1_score,
          roc_auc_score,
          confusion_matrix
      )
```

2. Load Local ARFF Dataset

```
[11]: data = load_arff('/Users/jacobsmith/Desktop/Data Science/Training Dataset.arff')
      df = pd.DataFrame(data)

      df.head()
```

	having_IP_Address	URL_Length	Shortining_Service	having_At_Symbol	double_slash_redirecting	Prefix_Suffix	having_Sub_Domain	SSLfinal_State	Domain_registeration_length	Favicon	...	popupWindow	Iframe	age_of_domain	DNSRecord	web_traffic
0	-1	1	1	1	1	-1	-1	-1	-1	1	...	1	1	-1	-1	0
1	1	1	1	1	1	-1	0	0	-1	1	...	-1	1	-1	-1	0
2	1	0	1	1	1	-1	-1	-1	-1	1	...	1	1	1	-1	1
3	1	0	1	1	1	-1	-1	-1	-1	1	...	1	1	1	-1	1
4	1	0	-1	1	1	-1	1	1	-1	1	...	-1	1	-1	-1	0

5 rows × 31 columns

3. Data Cleaning Check

```
[12]: print("Missing values:\n", df.isnull().sum())
      print("Duplicate rows:", df.duplicated().sum())
      df = df.drop_duplicates()
      print("Shape after cleaning:", df.shape)

      print(df['Result'].value_counts())
      print("Phishing websites: 1")
      print("Legitimate websites: -1")
      print("Missing values: 0")
```

```
Missing values: 0
Duplicate rows: 0
Shape after cleaning: (11055, 32)
Result
 1    6157
-1    4898
Name: count, dtype: int64
Phishing websites: 1
Legitimate websites: -1
Missing values: 0
```

4. Feature Definitions

```
[13]: print("Number of features:", df.shape[1] - 1)
      print("Number of observations:", df.shape[0])
      print(df.dtypes)
```

5. Define Features and Target

```
[14]: X = df.drop('Result', axis=1)
      y = df['Result'].map({-1: 0, 1: 1})
      print("Feature shape:", X.shape)
      print("Target shape:", y.shape)
      print("Target distribution:\n", y.value_counts())
```

```
Feature shape: (11055, 30)
Target shape: (11055,)
Target distribution:
 1    6157
0    4898
Name: count, dtype: int64
```

6. Train/Test Split

```
[15]: from sklearn.model_selection import train_test_split

      X_train, X_test, y_train, y_test = train_test_split(
          X, y,
          test_size=0.2,
          random_state=42,
          stratify=y
      )

      print("Training set:", X_train.shape)
      print("Testing set:", X_test.shape)
      print("Training set: 80%")
      print("Testing set: 20%")
```

```
Training set: (8844, 30)
Testing set: (2211, 30)
Training set: 80%
Testing set: 20%
```

7. Initial Model

```
[16]: models = {
          "Logistic Regression": LogisticRegression(max_iter=1000),
          "Decision Tree": DecisionTreeClassifier(random_state=42),
          "Random Forest": RandomForestClassifier(random_state=42)
      }

      for name, model in models.items():
          model.fit(X_train, y_train)
          print(f"{name} trained successfully.")
```

```
Logistic Regression trained successfully.
Decision Tree trained successfully.
Random Forest trained successfully.
```

8. Final and Evaluate Models

```
[17]: results = {}

      for name, model in models.items():
          y_pred = model.predict(X_test)
          y_prob = model.predict_proba(X_test)[:, 1]

          results[name] = {
              "Accuracy": accuracy_score(y_test, y_pred),
              "Precision": precision_score(y_test, y_pred),
              "Recall": recall_score(y_test, y_pred),
              "F1-Score": f1_score(y_test, y_pred),
              "ROC-AUC": roc_auc_score(y_test, y_prob)
          }

          print(f"\n{name} Results:")
          print("Accuracy:", results[name]["Accuracy"])
          print("Precision:", results[name]["Precision"])
          print("Recall:", results[name]["Recall"])
          print("F1-Score:", results[name]["F1-Score"])
          print("ROC-AUC:", results[name]["ROC-AUC"])
          print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
          print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
Logistic Regression Results:
Accuracy: 0.9312528268655814
Precision: 0.9387755102040817
Recall: 0.9441287878787878
F1-Score: 0.9414442700156985
ROC-AUC: 0.9799139167862267

Confusion Matrix:
 [[ 894   70]
 [  65 1182]]

Classification Report:
              precision    recall  f1-score   support

           0       0.93      0.93      0.93       964
           1       0.94      0.94      0.94      1247

    accuracy                           0.93      2211
   macro avg       0.93      0.93      0.93      2211
weighted avg       0.93      0.93      0.93      2211


Decision Tree Results:
Accuracy: 0.9565807327001356
Precision: 0.9624
Recall: 0.9631036914995987
F1-Score: 0.9627516778523489
ROC-AUC: 0.9565807327001356

Confusion Matrix:
 [[ 922   42]
 [  41 1206]]

Classification Report:
              precision    recall  f1-score   support

           0       0.96      0.96      0.96       964
           1       0.97      0.96      0.96      1247

    accuracy                           0.96      2211
   macro avg       0.96      0.96      0.96      2211
weighted avg       0.96      0.96      0.96      2211


Random Forest Results:
Accuracy: 0.9737220269560379
Precision: 0.9776
Recall: 0.9767442406732158
F1-Score: 0.9771719038817006
ROC-AUC: 0.9966
```

```
[[569  35]
 [ 31 535]]

Classification Report:
              precision    recall  f1-score   support

          -1       0.95      0.94      0.95       604
           1       0.94      0.95      0.94       566

    accuracy                           0.94      1170
   macro avg       0.94      0.94      0.94      1170
weighted avg       0.94      0.94      0.94      1170
```

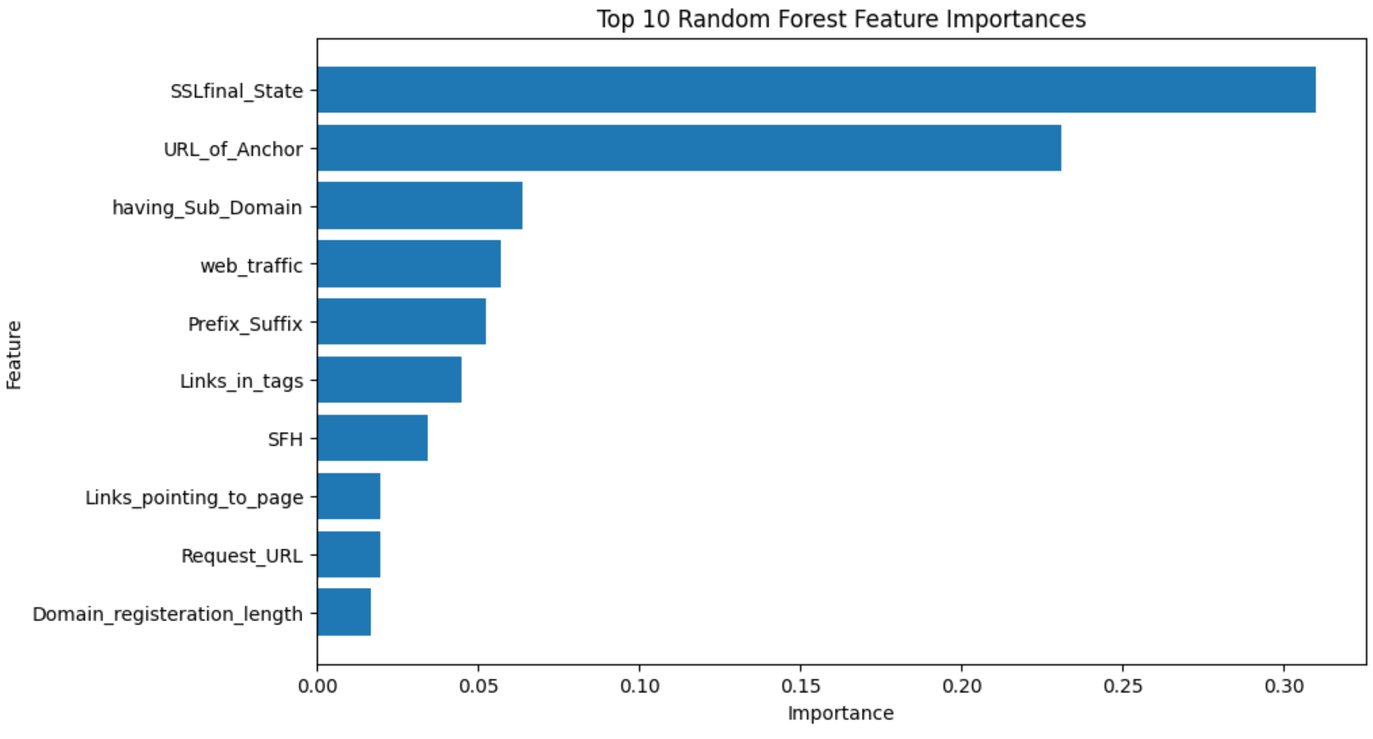
In [9]:
```python
# 9. Model Comparison Table

results_df = pd.DataFrame(results)

results_df = results_df.sort_values(
    by="F1 Score",
    ascending=False
)

results_df
```

Out[9]:

	Model	Accuracy	Precision	Recall	F1 Score
2	Random Forest	0.943590	0.938596	0.945230	0.941901
0	Logistic Regression	0.912821	0.900000	0.922261	0.910995
1	Decision Tree	0.909402	0.888514	0.929329	0.908463

In [10]:
```python
# 10. Random Forest Feature Importance

rf_model = models["Random Forest"]
rf_model.fit(X_train, y_train)

feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": rf_model.feature_importances_
})

feature_importance = feature_importance.sort_values(
    by="Importance",
    ascending=False
)

feature_importance.head(10)
```

Out[10]:

	Feature	Importance
7	SSLfinal_State	0.310261
13	URL_of_Anchor	0.230988
6	having_Sub_Domain	0.063919
25	web_traffic	0.056940
5	Prefix_Suffix	0.052511
14	Links_in_tags	0.044871
15	SFH	0.034145
28	Links_pointing_to_page	0.019774
12	Request_URL	0.019592
8	Domain_registeration_length	0.016483

In [11]:
```python
# 11. Plot Top 10 Important Features

top_features = feature_importance.head(10)

plt.figure(figsize=(10, 6))
plt.barh(top_features["Feature"], top_features["Importance"])
plt.gca().invert_yaxis()
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.title("Top 10 Random Forest Feature Importances")
plt.show()
```